Transfert Patients — Documentation technique

Oct 15, 2026 · @Someone

Inventaire des artefacts

Tous les artefacts appartiennent au scope `x_2218203_ptrans` (application `f17d5cef93fb8310668230018bba10a9`) ; le code source est versionné dans `TransfertPatients/src/`.

Type	Nombre	Éléments
Tables	8	`_specialite`, `_etablissement`, `_service`, `_demande` (étend `task`), `_reponse`, `_relance`, `_reservation`, `_regle_equipement`
Rôles	4	`admin`, `regulateur`, `coordinateur`, `demandeur`
Script Includes	3	`TransfertUtils`, `TransfertPortail`, `TransfertAjax` (client-callable)
Business Rules	10	9 sur les tables de l'application, 1 sur `wm_order`
UI Actions	6	Publier, Annuler la demande, Republier, Accepter, Refuser, Retenir cette destination
Job planifié	1	Transfert patients - expiration des demandes (1 min)
Notifications	6	Nouvelle demande, réponse, destination confirmée, non retenue, expirée, relance d'un établissement
Fix Script	1	Transfert Patients - créer les ACL (scope global, ACL rattachées à l'application)
Record producer	1	Demande de transfert de patient (24 variables, 3 client scripts, 1 user criteria)
Service Portal	1	Portail `transferts`, thème, en-tête, 8 widgets, 5 pages, 1 feuille de style, 1 dépendance (Leaflet)
Interface classique	—	Menu de 22 modules, formulaires, listes, barre d'étapes

Type	Nombre	Éléments
Données FSM	—	3 zones `cmn_location`, 12 véhicules sanitaires alm_asset, 7 groupes (`wm_dispatch`, `wm_work`, `wm_qualification`)
Définitions de SLA	4	Transfert - réponse - vitale (5 min) / urgente (15 min) : début urgence + state=10, arrêt nb_acceptations>0 OU state=4, annulation state=7 ; Transfert - prise en charge - vitale (10 min) / urgente (15 min) : début state IN 20,30,40, arrêt state=3, annulation state=7. Sans planning, fuseau GMT, sans flux

Utilisateurs créés dans `sys_user` : 16 utilisateurs de démo (régulateur, 3 coordinateurs, 2 agents, 3 dispatchers, 6 ambulanciers) et 2 comptes d'intégration, `diapandal` (actif pour l'API et le job) et `adminclaude` (conservé). Les rôles métier sont attribués directement ou hérités des groupes de coordination, de dispatch, d'équipages et de qualification.

Tables et champs

Préfixe des tables : `x_2218203_ptrans_`. Les listes de choix sont définies en français et en anglais.

Table	Champs
`specialite`	`nom` (affiché), `code`, `description`, `active`, `niveau_soins_defaut`, `mode_transport_defaut`, `equipements_defaut` (liste `cmdb_model`)
`etablissement`	`nom`, `type_etablissement`, `location` (`cmn_location`), `code_etablissement`, `telephone_regulation`, `coordinateur` (`sys_user`), `groupe_coordination` (`sys_user_group`), `capacite_totale` et `lits_disponibles` (calculés, lecture seule), `accepte_transferts`, `active`
`service`	`nom`, `etablissement`, `specialite`, `lits_totaux`, `lits_disponibles`, `responsable` (`sys_user`), `active`

Table	Champs
`demande` (TRF, étend `task`)	`patient_reference`, `patient_age`, `patient_sexe`, `etablissement_origine`, `service_origine`, `specialite_requise`, `niveau_soins`, `urgence_transfert` (1, 2, 3), `mode_transport`, `mobilite`, `equipements_requis` (liste `cmdb_model`), `date_souhaitee`, `telephone_contact`, `date_limite_reponse`, `etablissement_destination`, `service_destination`, `reponse_retenue`, `work_order` (`wm_order`), `nb_eligibles`, `nb_acceptations`, vehicule (alm_asset), demande_precedente et republication (demande), date_publication, date_premiere_acceptation, date_retenue, date_limite_arrivee, date_arrivee, itineraire (polyline, 4000), distance_route_km, duree_route_min ; `state` avec 8 valeurs propres (1, 10, 20, 30, 40, 3, 4, 7)
`reponse` (REP)	`number`, `demande`, `etablissement`, `decision` (en_attente, acceptee, refusee, retenue, non_retenue, expiree), `service_propose`, `motif_refus`, `commentaire`, `heure_arrivee_possible`, `date_reponse`, `repondu_par`, distance_km, duree_route_min
`relance` (RLC)	number, demande, etablissement, service_propose, commentaire, statut (envoyee, traitee), demande_par, date_relance
`reservation` (RES)	number, asset (alm_asset), type_reservation (equipement, vehicule), demande, etablissement (vide pour un véhicule), statut (active, liberee), date_debut, date_fin, motif_liberation
`regle_equipement`	modele (cmdb_model, affiché), quantite_minimum (défaut 1), active

La numérotation utilise `sys_number` (préfixes TRF, REP, RLC et RES, 7 chiffres). Le défaut de `state` sur la demande est surchargé à 1 (Brouillon) par `sys_dictionary_override`.

Script Includes

`TransfertUtils` — règles métier, appelé par les Business Rules, le job et les deux autres Script Includes.

Méthode	Contrat
`publier(demande)`	Appelée en before ; crée une réponse « en_attente » par établissement éligible, renseigne `nb_eligibles`, `date_limite_reponse` (si vide) et une note ; retourne le nombre d'établissements. Ne fait pas `update()`.
`retenir(reponse)`	Après passage de la décision à « retenue » : lit – 1, reserverEquipements(), autres réponses « non_retenue », destination sur la demande (état 20), puis `creerOrdreTransport()`
`creerOrdreTransport(demande)`	Crée `wm_order`, avec le véhicule réservé par affecterVehicule() dans sa description, le relie aussitôt à la demande (état 30, champ vehicule), crée la tâche unique « Transport du patient » (`wm_task`, puis applique le premier State Flow valide de `FLOWS_WO_DEPART`. Tâches et transitions sont isolées dans des try/catch, car une règle FSM peut lever une exception pour un utilisateur sans rôle FSM. Appelle ensuite assignerTaches() (try/catch). Retourne le sys_id du WO.
`synchroniserDepuisWorkOrder(wo)`	18 → demande 40 ; 3 → demande 3 ; 4 ou 7 → note d'alerte
`recalculerCapacite(etabId)`	Somme des lits des services actifs vers l'établissement
`compterAcceptations(demandeId)`	Met à jour `nb_acceptations` (acceptee + retenue)
`expirerDemandes()`	Demandes à l'état 10, date limite dépassée, 0 acceptation → état 4, réponses en attente → expiree
`estCoordinateur(etabId)`	Vrai si l'utilisateur est le coordinateur ou membre du groupe de coordination

Méthode	Contrat
`mesEtablissements()`	sys_ids des établissements coordonnés, séparés par des virgules (filtres de listes)
`contexteDemandeur(userId)`	`{etablissement, service, telephone, source}` : responsable de service, puis coordination, puis localisation (et parents)
`suggestionsSpecialite(specialiteId)`	`{niveau_soins, mode_transport, equipements, equipements_noms}` lus sur la spécialité
`republier(ancienne)`	Demande à l'état 4 et non republiée : copie les champs, insère en état 1 puis passe à 10 (publication), relie demande_precedente / republication, passe les relances en traitee ; retourne la nouvelle demande ou null
`reserverEquipements(demande, etabId)`	Une réservation « equipement » par modèle requis, sur la première unité libre de l'établissement ; retourne le texte de la note (réservés, et manquants s'il n'y a plus d'unité libre)
`affecterVehicule(demande)`	Modèles compatibles par mode (_modelesVehicule) ; cherche dans la localisation de l'origine et ses parents, puis partout ; crée une réservation « vehicule » ; retourne l'asset ou null
`libererReservations(demande, motif)`	Réservations actives → liberee, date_fin, motif_liberation ; retourne leur nombre
`_unitesDisponibles(loc, modeles, max)`	Assets des modèles à la localisation (null = toutes), install_status 1 ou 6, sous-statut hors in_repair, defective, pending_repair, reserved, sans réservation active ; au plus max. Utilisée par _equipementsDisponibles() avec _quantiteMinimum() (règle active du modèle, sinon 1)

Méthode	Contrat
`assignerTaches(demande, tacheIds, vehicule)`	Équipage = _equipageDe(localisation du véhicule, sinon de l'origine) ; agent = _ambulancierDisponible(groupe), le membre qui a le moins de wm_task en état 10, 12, 16, 17, 18. Pour chaque tâche, dans l'ordre : expected_start enchaîné, estimated_work_duration (route + REMISE_MIN 2 min ; programmée : TRAVAIL_PROGRAMME_MIN 90), estimated_travel_duration (approche = temps restant dans la fenêtre, APPROCHE_MIN 2 ; programmée : 30), assignment_group, assigned_to. Note sur la demande avec le nombre de tâches assignées
`_equipageDe(locationId)`	Premier groupe actif de type wm_work (TYPE_GROUPE_EQUIPAGE) qui couvre la localisation ou un parent (sys_group_covers_location)
`distanceKm(locA, locB)`, `dureeRouteMin(km)`	Haversine sur latitude / longitude de cmn_location, arrondi à 0,1 km, null si une coordonnée manque ; durée = km × FACTEUR_ROUTE (1,3) / VITESSE_KMH (25) × 60, minimum 5 min. Appelées par publier() pour chaque réponse ; TransfertPortail._distance() formate « 3,2 km · ≈ 10 min de route », detail() trie les réponses par distance et marque plusProche

Méthode	Contrat
`delaiMin(type, urgence)`	Durée en minutes de la définition de SLA active (SLA_NOMS), sinon DELAIS_DEFAUT (réponse 5/15/1440, prise en charge 10/15) ; null si aucun délai. Utilisée par _dateLimite(), retenir() (date_retenue, date_limite_arrivee) et assignerTaches() : avec une échéance, transport = duree_route_min de la réponse retenue (⩾ 3), travail = transport + 2 min de remise, approche = reste (⩾ 2), expected_start = maintenant + approche, window_start / window_end = retenue → échéance (fin planifiée si intenable, avec note) ; sans échéance, DUREES_TACHES et TRAJETS_TACHES. publier() pose date_publication, compterAcceptations() date_premiere_acceptation, synchroniserDepuisWorkOrder() date_arrivee
`TransfertPortail._minuteur(d)`	Ajouté à _resume() (mt) : phase reponse | decision | arrivee | bilan | fige, sens rebours | chrono | fige, debutMs, echeanceMs, titre, sous, bilan [{label, texte, cible, ok}]. Les widgets tp-demande, tp-regulation et tp-arrivees l'animent avec une fonction vue() (rouge sous 1 min, dépassement « +m:ss ») ; detail() ajoute estimMin et delaiPec à chaque réponse
`calculerItineraire(locA, locB)`	GET {osrm_url}/route/v1/driving/lonA,latA;lonB,latB?overview=full&geometries=polyline (simplified si > 3 990 caractères), délai 8 s ; retourne {polyline, km, minutes = durée × coefficient_circulation, 3 min minimum} ou null. Appelée par retenir() ; assignerTaches() prend duree_route_min de la demande pour le Transport. Propriétés x_2218203_ptrans.itineraire.osrm_url et .coefficient_circulation (créées par build/10_delais_sla.py). TransfertPortail._trajet() alimente la carte du widget tp-demande (décodage polyline, position estimée le long du tracé, dépendance Leaflet)

`TransfertPortail` — données et actions des widgets ; le constructeur calcule l'utilisateur, le rôle régulateur et ses établissements.

Méthode	Usage
`contexte()`, `accueil()`, `aTraiter()`, `mesDemandes(n)`, `capacite()`, `arrivees()`	Page d'accueil et en-tête
`liste(onglet)`	Onglets recues, emises, historique
`detail(demandeId)`	Détail avec étapes, parcours, transport (véhicule et équipements réservés), compte à rebours (limiteMs, maintenantMs), demande précédente ou republication, relances, réponse de l'utilisateur ; toutes les réponses pour le régulateur ; `{interdit:true}` si non autorisé
`regulation()`	Colonnes, indicateurs, lits libres par spécialité ; réservé au régulateur
`repondre(p)`, `ajusterLits(id, delta)`, `retenir(id)`, `changerEtat(id, etat)`, republier(id), relancer(p)	Actions ; vérifient les droits et retournent `{ok, message}`
`_transport(d)`	Suivi utilisé par detail() et arrivees() : numero, etat, agent, groupe, telephone (si detaille), vehicule, equipements, courante (rang de la première tâche non terminée), retardMin, eta / etaMs, termine, maintenantMs ; par tâche : numero, nom, lieu, etat, statut (fait, courant, futur, annule), prevu / reel (HH:mm), prevuDebutMs, prevuFinMs, reelDebutMs, reelFinMs, dureeMin, asset (wm_task.asset, sinon véhicule pour le transport), etapes (Assignée, Acceptée, En route, Démarrée, Terminée avec heure), note {auteur, heure, texte}

Méthode	Usage
`_heuresEtats(tacheId)`, `_derniereNote(t)`	Premier passage dans chaque état lu dans sys_audit (fieldname state) ; dernière entrée work_notes (getJournalEntry(1)) découpée en auteur, heure, texte. Helpers _ms, _dureeMs, _hm (HH:mm au fuseau de l'utilisateur)
`missions()`, `actionMission(demandeId, code, note)`	Missions = Work Orders où l'utilisateur a des wm_task ouvertes ou clôturées aujourd'hui ; _prochaineAction() déduit l'action des états (accepter, partir, sur_place, prise_en_charge, arrive, terminer ; secondaires refuser, incomplete). actionMission() revérifie l'action, exige une note pour terminer, refuser et incomplete, puis _flux() applique le state flow FLOWS_TACHE (Accept, Reject, Start Travel, Start Work, Close Complete, Close Incomplete) sur une tâche assignée à l'utilisateur, la note dans la même mise à jour, et contrôle que l'état a changé
`Missions à tâche unique`	_prochaineAction() : 16 → accepter (+ refuser) ; 17 sans actual_travel_start → partir (Start Travel) ; 17 sans actual_onsite_arrival → sur_place (Check-In, l'heure d'arrivée sur site est posée par l'application) ; 17 → prise_en_charge (Start Work) ; 18 → terminer (Close Complete, note) + incomplete. Jalons affichés : en route, sur place, pris en charge, remis. Les ordres à 3 tâches gardent l'ancien enchaînement

`TransfertAjax` (client-callable, étend `global.AbstractAjaxProcessor`) : `getContexte()` et `getSuggestions(sysparm_specialite)` renvoient du JSON au formulaire.

Business Rules, UI Actions, job et notifications

Business Rule	Table	Quand	Condition	Effet
Demande - validation	demande	before ins/upd, 100	—	Service d'origine cohérent, description courte, `active` selon l'état
Demande - publication	demande	before ins/upd, 200	`state.changesTo(10)`	`publier(current)`
Demande - clôture des réponses	demande	after upd	état → 7 ou 4	Réponses ouvertes → non_retenue / expiree
Réponse - contrôle décision	reponse	before upd, 100	`decision.changes()`	Valide acceptée, refusée, retenue ; horodate
Réponse - compteur acceptations	reponse	after upd, 100	`decision.changes()`	`compterAcceptations()`
Réponse - destination retenue	reponse	after upd, 200	`decision.changesTo('retenue')`	`retenir(current)`
Service - contrôle lits	service	before ins/upd	—	0 ⩽ libres ⩽ totaux
Service - capacité établissement	service	after ins/upd/del	lits, actif ou établissement modifiés	`recalculerCapacite()`
Transfert patient - suivi ordre FSM	`wm_order`	after upd, 200	`state.changes()`	`synchroniserDepuisWorkOrder()`
Demande - libération des réservations	demande	after upd, 150	état → 3, 7 ou 4	`libererReservations(current, motif)`

UI Action	Table	Condition d'affichage
Publier	demande	état 1 et (régulateur ou auteur)
Annuler la demande	demande	état 1, 10 ou 20 et (régulateur ou auteur)
Accepter / Refuser	reponse	décision ouverte, demande publiée, coordinateur de l'établissement ou admin
Retenir cette destination	reponse	acceptee, demande publiée, régulateur
Republier	demande	état 4, republication vide et (régulateur ou auteur)

- **Job** « Transfert patients - expiration des demandes » : `sysauto_script`, périodique 1 min, appelle `expirerDemandes()`.
- **Notifications** : `sysevent_email_action` déclenchées sur insertion ou mise à jour des tables reponse et demande ; destinataires par champs (`etablissement.groupe_coordination`, `etablissement.coordinateur`, `demande.opened_by`).

Portail

Portail `sp_portal` `transferts` (sys_id `24fb556f93f3c310668230018bba1079`), nom affiché « Waluu » (titre, en-tête, thème, préfixe « Waluu · » des pages et des widgets ; renommé le 15/10/2026 par build/migrations/20261015_renommage_waluu.py, l'application et le menu gardent le nom Transfert Patients), thème « Waluu » : variables SCSS de marque, feuille `sp_css` partagée (classes `tp-*`), police Lato, en-tête `tp-header`. Pages non publiques, réservées aux 4 rôles de l'application.

Page	Disposition	Widgets
`tp_accueil`	conteneur fluide 12 + conteneur 8 / 4	`tp-accueil-hero` ; `tp-a-traiter`, `tp-mes-demandes` ; `tp-capacite`, `tp-arrivees`
`tp_demandes`	12	`tp-liste`
`tp_demande`	12, paramètre `sys_id`	`tp-demande`
`tp_regulation`	conteneur fluide 12	`tp-regulation`
tp_carte	conteneur 12	tp-carte
tp_missions	conteneur 12, rôles de l'application + ambulancier	tp-missions (dépendance Leaflet) ; tp-header redirige un ambulancier seul vers cette page

Widget	Données serveur	Actions (`input.action`)	`recordWatch`
`tp-header`	`contexte()`, liens selon les rôles	—	—
`tp-accueil-hero`	`accueil()`	—	demande, reponse, service
`tp-a-traiter`	`aTraiter()`, motifs	`repondre`	reponse, service
`tp-mes-demandes`	`mesDemandes(6)`	—	demande
`tp-capacite`	`capacite()`	`ajuster`	service
`tp-arrivees`	`arrivees()`	—	demande, wm_task

Widget	Données serveur	Actions (`input.action`)	`recordWatch`
`tp-demande`	`detail(sys_id)`	`repondre` , `retenir` , `etat`	demande (sys_id), reponse (demande), wm_task
`tp-liste`	`liste()` × 3 onglets	— (filtres côté client)	demande, reponse
`tp-regulation`	`regulation()`	—	demande, reponse, service
tp-carte	carte() : hopitaux (lat, lon, services, equipements libres, mien), zones (ambulances libres au centre des hôpitaux de la zone), transferts (de, vers, mt), detaille ; dépendance « Leaflet 1.9.4 » (cdnjs, JS + CSS), tuiles OpenStreetMap	— (filtres côté client)	service, demande, reservation

Convention : `controllerAs = c` ; les actions passent par `c.server.get({action, …})` , le serveur renvoie `data.resultat = {ok, message}` affiché avec `spUtil.addInfoMessage` / `addErrorMessage` ; les confirmations utilisent `spModal.confirm` . Le code est dans `src/portal/widgets/<id>/` (`template.html` , `style.scss` , `client.js` , `server.js`).

Record producer

`sc_cat_item_producer` « Demande de transfert de patient » (`346bb8af93ff8310668230018bba10fc`), table `x_2218203_ptrans_demande` , catégorie « Transferts de patients » du Service Catalog, user criteria « Transfert Patients - demandeurs » (rôles regulateur, admin, demandeur, coordinateur).

Conteneur	Variables mappées (nom = champ)
Origine et urgence	`etablissement_origine` , `service_origine` (qualificateur `etablissement=` + attribut `ref_qual_elements`), `urgence_transfert` , `date_souhaitee` , `telephone_contact`

Conteneur	Variables mappées (nom = champ)
Patient	`patient_reference`, `patient_age`, `patient_sexe`, `mobilite`
Besoin médical et transport	`specialite_requise`, `niveau_soins`, `mode_transport`, `equipements_requis` (list collector `cmdb_model`)
—	`description` (mappée), `publier_immediatement` (case non mappée)

Les listes déroulantes utilisent `choice_table` / `choice_field` de la demande.

Script	Rôle
Script (avant insertion)	`opened_by`, état 1, nettoyage de l'âge, redirection vers la demande
Post-insert script	Si la case est cochée : état 10 + `update()`, ce qui déclenche la publication
onLoad « Pré-remplir selon le profil »	GlideAjax `getContexte` → établissement, service, téléphone, message d'information
onChange `specialite_requise`	GlideAjax `getSuggestions` → niveau, mode, équipements
onChange `etablissement_origine`	Vide le service d'origine

Deux points de mise en œuvre : le champ `cat_variable` des scripts onChange (valeur `IO:<sys_id de la variable>`) est une liste de choix dynamique que l'API Table refuse ; `build/07_record_producer.py` l'écrit côté serveur avec `sn.run_script()`. Après envoi depuis le portail, `producer.portal_redirect` ouvre la page `tp_demande` de la demande créée.

Build et déploiement

Le déploiement passe par l'API REST Table avec le client `tools/sn.py` (`get`, `post`, `patch`, `delete`, `upsert`, `ensure`). Les écritures atterrissent dans le scope grâce à la préférence utilisateur `apps.current_app` = application. Chaque script est idempotent (upsert sur une clé métier). sn.run_script() réessaie l'appel quelques secondes quand l'API temporaire n'est pas encore enregistrée (HTTP 400).

Dépendances externes (gratuites, sans clé d'API) :

Dépendance	Version ou adresse	Utilisée par	Configuration
Leaflet	1.9.4, cdnjs (`leaflet.min.js` , `leaflet.min.css`)	Widgets `tp-carte` , `tp-demande` , `tp-missions`	Dépendance de widget « Leaflet 1.9.4 » (`sp_dependency`)
Fonds de carte OpenStreetMap	`https://tile.openstreetmap.org/{z}/{x}/{y}.png`	Mêmes widgets	Option `referrerPolicy: 'strict-origin-when-cross-origin'` obligatoire
OSRM	`https://router.project-osrm.org` (profil voiture)	`TransfertUtils.calculerItineraire()` à la retenue	Propriétés `x_2218203_ptrans.itineraire.osrm_url` et `.coefficient_circulation`

Outils de développement : Python 3 (scripts `build/01` à `10` , `build/migrations`), client REST `tools/sn.py` (Table API, `run_script()` par API REST temporaire avec nouvelles tentatives), Git, python-pptx (présentations), Claude Docs (documentations exportées en PDF dans `docs/livrables`).

Ordre	Script	Déploie
1	`build/01_schema.py`	Rôles, tables, champs, choix, états, numérotation
2	`build/02_logique.py`	3 Script Includes, 10 Business Rules, 6 UI Actions, job
3	`build/03_donnees_demo.py`	Spécialités et défauts, hôpitaux, services, équipements, groupes, utilisateurs de démo, règles d'équipement et quantités (réinitialise les lits) ; --utilisateurs ou --equipements pour ne déployer que ces données, sans toucher aux lits
4	`build/04_interface.py`	Formulaires, listes, barre d'étapes, menu
5	`build/05_securite_notifications.py`	Fix Script ACL, notifications
6	`build/06_dispatch.py`	Zones, groupes FSM, véhicules sanitaires, dépendances dispatch → équipages (sm_m2m_group_dependency) et couverture régionale des équipages
7	`build/07_record_producer.py`	Record producer, variables, client scripts, user criteria

Ordre	Script	Déploie
8	`build/08_portail.py`	Thème, en-tête, widgets, pages (mise en page recréée), portail
9	`build/09_scenarios_demo.py`	Demandes de démo à chaque étape
10	`build/10_delais_sla.py`	Définitions de SLA des délais (flux par défaut retiré)

```
cd TransfertPatients && for s in build/0*.py; do python3 "$s" || break; done
```

Prérequis : `.env.servicenow` à la racine du repo (`SN_INSTANCE` , `SN_USER` , `SN_PASS`), utilisateur API admin, plugin Field Service Management actif. Après le déploiement, et après chaque nouvelle table (dernières : relance, reservation, regle_equipement), exécuter le Fix Script ACL en `security_admin` .

Compte d'intégration : `diapandal` (rôles `admin` et `snc_basic_auth_api_access`), avec la préférence `apps.current_app` sur l'application. Deux cas particuliers : le champ `cat_variable` des scripts onChange (07) est écrit par `sn.run_script()` , qui exécute un script serveur via une Scripted REST API temporaire créée puis supprimée ; le Fix Script ACL (05) est créé en scope global en basculant temporairement `apps.current_app` sur global.

Tests

Le code serveur est testé sans navigateur de deux façons : une Scripted REST API temporaire, créée puis supprimée, qui exécute un script dans le scope ; et l'API `/api/now/sp/page` , qui exécute les scripts serveur des widgets d'une page.

Test	Résultat
Publication : spécialité, lits (service à 0 exclu), équipements (couveuse, respirateur)	Conforme
Refus sans motif	Bloqué
Retenue : lit − 1, autres réponses non retenues, WO + 3 tâches	Conforme
Transition FSM avec qualification active	WO en *Awaiting Qualification*
Groupes de qualification et de dispatch renseignés par FSM selon la zone	Conforme

Test	Résultat
Record producer soumis par l'API Service Catalog, publication immédiate	Conforme
Contexte du demandeur (localisation, responsable de service, coordination)	Conforme pour les 3 cas
Notifications générées vers les bons coordinateurs	Conforme
Scripts serveur des 9 widgets sur les 4 pages	Sans erreur
Accès refusé à une demande d'un autre établissement	`interdit`
Règle de quantité (15/10, TRF0001025) : Dalal Jamm, 1 respirateur pour un minimum de 2, exclu ; Fann et Idrissa Pouye sollicités	Conforme
Retenue de Fann : FANN-RESP-1, FANN-MONI-1 et VEH-PLATEAU-SMUR-1 (zone d'origine) réservés, véhicule dans WO0010017 et sur la demande	Conforme
Annulation : 3 réservations libérées avec date et motif ; detail() renvoie véhicule et équipements	Conforme (données de test supprimées ensuite)
Assignation directe (15/10, TRF0001028, Abass Ndao → Fann, inter-zones) : 3 tâches Assigned à Ambulancier Dakar Plateau-Médina 2, 16:02 → 16:32 → 17:32 ; WO passé en Assigned par FSM	Conforme (données de test supprimées)
Suivi des tâches (15/10, TRF0001030) : tâches avancées par les state flows FSM (Accept, Start Work) ; detail() renvoie prévu, réel, étapes horodatées, asset du transport, dernières notes, arrivée estimée ; widgets tp-demande et tp-arrivees exécutés sans erreur via /api/now/sp/page	Conforme (rendu visuel à vérifier dans le navigateur)

Test	Résultat
Distance (15/10, TRF0001034, médecine depuis Dalal Jamm) : 4 réponses de 8 km (Idrissa Pouye, ≈ 25 min) à 13,7 km (Principal, ≈ 43 min), triées ; « Le plus proche » sur Abass Ndao, accepté le plus proche ; 38 réponses existantes complétées	Conforme (données de test supprimées)
Délais vitale (15/10, TRF0001037, Abass Ndao → Fann) : limite réponse +5 min, réponse en 1:09 (SLA terminé, non dépassé), arrivée exigée +10 min, tâches 2 + 5 + 2 min dans la fenêtre 13:31 → 13:41, arrivée simulée : bilan 1:09 / 5:00 ✓ et 2:34 / 10:00 ✓, véhicule libéré	Conforme
Urgente (TRF0001038, Abass Ndao → Principal) : limites +15 / +15 min, tâches 2 + 10 + 2 min dans la fenêtre ; vitale intenable (TRF0001039, Dalal Jamm → Fann, ≈ 43 min) : repérée avant la retenue, tâches sur la durée réelle, note d'alerte ; expiration vitale sans réponse (TRF0001036) : Expirée 25 s après l'échéance, SLA réponse en dépassement	Conforme (données de test supprimées)
Itinéraire (15/10, TRF0001040, Idrissa Pouye → Principal, urgente) : 9,5 km par la route, 10 min OSRM × 1,5 = 15 min, tâche Transport de 15 min, délai 15 min signalé intenable ; detail().trajet complet (tracé 494 car., véhicule, lien OpenStreetMap) ; appel OSRM depuis l'instance en moins d'1 s	Conforme côté serveur (rendu de la carte à vérifier dans le navigateur)
Mission complète en tant qu'amb.fann2 (16/10, TRF0001040, session impersonée) : page chargée avec la mission à accepter ; action hors séquence refusée ; accepter, partir, sur place, prise en charge, arrivée, terminer sans note refusé, terminer avec note ; WO0010027 clôturé, demande Arrivé, véhicule libéré, SLA terminés	Conforme après correctif (note et clôture dans la même mise à jour)

Test	Résultat
Tâche unique (16/10, TRF0001043, vitale, Abass Ndao → Fann) : WOT0010084 « Transport du patient », approche 2 + route 5 + remise 2 min dans la fenêtre 10 min ; mission en tant qu'amb.plateau1 : accepter, je pars (Start Travel), sur place (Check-In + actual_onsite_arrival), prise en charge (Start Work), terminer sans note refusé, avec note accepté ; ordre clôturé, demande Arrivé, prise en charge 3:20 / 10:00 ✓	Conforme (données de test supprimées)

Non testé automatiquement : le rendu et les scripts client du portail et du formulaire, à vérifier dans un navigateur en se faisant passer pour `coord.albertroyer`, `reg.demo` et `agent.dalaljamm`.

Démonstration en direct du 14 octobre 2026 (navigateur, *Impersonate*) :

Test	Utilisateur	Résultat
Accueil, cartes à traiter, capacité, arrivées	`coord.albertroyer`	Conforme, temps réel
Acceptation de TRF0001013 depuis le portail	`coord.albertroyer`	Conforme ; visible aussitôt dans le tableau de régulation
Retenue de destination (TRF0001015, 0001013, 0001014)	`reg.demo`	Défaut corrigé : ordre non relié ; conforme après correctif (WO0010015)
Lecture des tables par profil après création des ACL	3 profils	Conforme aux règles (agent : ses demandes seulement)
Formulaire pré-rempli et suggestions par spécialité	`agent.dalaljamm`	Défaut corrigé : onChange inactif ; conforme après correctif
Envoi du formulaire et publication	`agent.dalaljamm`	TRF0001017 publiée à 2 établissements éligibles

Exploitation et dépannage

Symptôme	Cause probable	Action
Un utilisateur non admin ne voit pas les formulaires ou le record producer bloque	ACL non créées	Exécuter le Fix Script ACL en `security_admin`
Le formulaire ne pré-remplit pas l'établissement	Utilisateur sans service, groupe de coordination ni localisation	Renseigner `sys_user.location` ou l'ajouter au groupe de coordination
Une demande n'est publiée à personne	Aucun service avec lit libre, ou équipement en quantité insuffisante (règle d'équipement, unités réservées ou en réparation)	Vérifier les lits et les assets `alm_asset` localisés sur l'établissement
« Accueil » du portail vide pour un coordinateur	Pas de rattachement à un établissement	Même action que ci-dessus
Le WO reste en Draft	Aucun State Flow de départ valide pour la configuration FSM	Vérifier `sm_config` et les State Flows `wm_order`
Lits incohérents après une annulation	Le lit n'est pas libéré automatiquement (limite connue) ; équipements et véhicule le sont	Corriger les lits libres du service
Demandes de démo expirées	Date limite dépassée (job 15 min)	Relancer `build/09_scenarios_demo.py`
Ordre de transport « Véhicule : à affecter par le dispatch »	Aucun véhicule libre d'un modèle compatible, ou mode héliporté	Voir Transport → Véhicules sanitaires et Réservations en cours
Un asset reste compté réservé	Réservation restée active (demande clôturée hors des états 3, 7, 4)	Passer la réservation en Libérée
« Unable to download app package » à l'installation d'une application du Store (ex. sn_fsm_disp_wrkspc)	Le PDI envoie ses identifiants à appstoreprod.service-now.com au lieu d'apprepo, ce qui est bloqué (403 dans syslog)	Installer depuis developer.servicenow.com (Manage instance) ; sinon reset du PDI après sauvegarde
Tâche non assignée, « aborted by Business Rule Populate Group - Dispatch/Work »	L'équipage ne dépend pas du groupe de dispatch de la tâche ou ne couvre pas sa localisation	Relancer build/06_dispatch.py (sm_m2m_group_dependency, couverture régionale)
Tâche non assignée, « aborted by Business Rule Date Checks »	Conflit de planning de l'agent (chevauchement, trajet par défaut d'1 h)	Vérifier expected_start, durations et les autres tâches de l'agent ; réaffecter depuis Field Service
Heures du portail décalées de 7 h	Utilisateur sans fuseau : le PDI applique America/Los_Angeles	time_zone = GMT sur l'utilisateur (fait par 03 et 06 pour la démo) ou fuseau système

Symptôme	Cause probable	Action
Un changement d'état de wm_task par l'API ou par script est ignoré	Les états FSM passent par les state flows ; Close Complete exige que l'utilisateur soit l'assigné	Utiliser global.StateFlow().processFlow() ou les boutons Field Service de l'ambulancier
Feuille de styles du portail en double après un redéploiement	Le nom d'une inclusion CSS locale est recopié depuis sa feuille sp_css : une recherche par nom ne la retrouve pas (corrigé le 15/10/2026)	08_portail.py cherche l'inclusion par source=local et sp_css ; supprimer une inclusion non liée au thème
Carte : tuiles « Access blocked »	Les serveurs OpenStreetMap refusent les tuiles demandées sans en-tête Referer ; la politique de ServiceNow (com.glide.security.referrerpolicy) ne l'envoie pas	Option Leaflet referrerPolicy: 'strict-origin-when-cross-origin' sur la couche de tuiles (corrigé le 15/10/2026) ; recharger la page sans cache
Clôture d'une tâche sans effet, sans erreur	sm_config mandatory_work_notes : la note doit être dans la même mise à jour que la transition	Poser work_notes sur l'enregistrement avant processFlow, sans update séparé (fait dans _flux)
Aucun email de notification n'arrive	Envoi des emails désactivé sur le PDI (glide.email.smtp.active = false) : les notifications restent dans la boîte d'envoi	Activer l'envoi des emails sur l'instance (décision de l'administrateur) ; le portail temps réel fonctionne sans email

Problèmes rencontrés pendant la démonstration, désormais corrigés :

Symptôme	Cause	Correctif
Aucun utilisateur non admin ne lit les tables	Fix Script scopé : insertions dans `sys_security_acl` refusées sans erreur	Fix Script en scope global avec compte rendu réel (29 ACL, 0 échec)
Demande en « Destination retenue » sans ordre de transport	Exception d'une règle FSM pour un utilisateur sans rôle FSM	Liaison du WO dès sa création ; tâches et transitions en try/catch avec `gs.warn`
Suggestions par spécialité absentes	`cat_variable` vide sur les scripts onChange	Écriture côté serveur par `sn.run_script()`
Page ticket standard après envoi	Pas de redirection portail	`producer.portal_redirect` vers `tp_demande`
Notes « Demande expirée » sous un compte d'intégration	Le job s'exécute sous l'utilisateur qui l'a créé	`run_as` = `diapandal`

Les journaux applicatifs sont dans `syslog` (source du scope) ; les Business Rules et Script Includes utilisent `gs.addErrorMessage` pour les refus visibles par l'utilisateur.